Monotonic Stack

> → **Key Idea**: Use stack that maintains elements in monotonic order to find next/previous greater/smaller elements

> • **When to use**: Next greater element, histogram problems, temperature problems, expression evaluation

> ⊕ **Complexity**: Time O(n), Space O(n)

> ⚠ **Common Pitfall**: Not handling empty stack or maintaining wrong monotonic order

Overview

Visual Example

Monotonic Stack Operations

```
Next Greater Element: [2, 1, 2, 4, 3, 1]

Step 1: Process 2        Step 2: Process 1        Step 3: Process 2
Stack: [2]              Stack: [2, 1]           Stack: [2]
                                                NGE[1] = 2

Step 4: Process 4        Step 5: Process 3        Step 6: Process 1
Stack: []               Stack: [4, 3]           Stack: [4, 3, 1]
NGE[2] = 4
NGE[2] = 4

Final Result: [4, 2, 4, -1, -1, -1]

>> Stack maintains decreasing order, pops when greater element found
```

Stack State Evolution

```
Input:  [2, 1, 2, 4, 3, 1]
        ↓

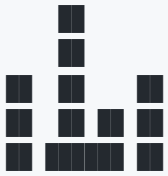
Step by step:
|     |    |     |    |  2  |    |  4  |    |  4  |    |  4  |
|     | -> |  2  | -> |  1  | -> |     | -> |  3  | -> |  3  |
|     |    |  1  |    |     |    |     |    |     |    |  1  |
+-----+    +-----+    +-----+    +-----+    +-----+    +-----+
  Empty      Add 2     Add 1     Pop all    Add 3,4    Add 1
```

```
>> Monotonic decreasing stack for Next Greater Element
```

Histogram Area Calculation

```
Heights: [2, 1, 5, 6, 2, 3]

Visual representation:
```



```
Largest rectangle: Height 5, Width 2 → Area = 10

Stack helps find left and right boundaries efficiently
```

A monotonic stack maintains elements in monotonic order (either strictly increasing or decreasing). It's used to solve problems that require finding the next or previous greater/smaller element efficiently.

The key insight is that when we encounter an element that violates the monotonic property, we can process all elements in the stack that are smaller/larger before adding the current element.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
```

```
// Template code for common variations
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity
Next/Previous Element	O(n)	O(n)
Daily Temperatures	O(n)	O(n)
Histogram	O(n)	O(n)
Expression Evaluation	O(n)	O(n)

Each element is pushed and popped at most once, giving O(n) amortized time complexity.

Common Problem Patterns

1. Next Greater Element

Problem: Find next greater element for each element in array.

```
public int[] nextGreaterElement(int[] nums) {
    int[] result = new int[nums.length];
    Arrays.fill(result, -1);
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

2. Daily Temperatures

Problem: Find days until warmer temperature for each day.

```
public int[] dailyTemperatures(int[] temperatures) {
    int[] result = new int[temperatures.length];
    Stack<Integer> stack = new Stack<>();
```

```
        for (int i = 0; i < temperatures.length; i++) {
            while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i]) {
                int prevDay = stack.pop();
                result[prevDay] = i - prevDay;
            }
            stack.push(i);
        }
        return result;
    }
```

3. Largest Rectangle in Histogram

Problem: Find largest rectangle area in histogram.

```
public int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int maxArea = 0;

    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];
        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

4. Trapping Rain Water

Problem: Calculate trapped rain water between bars.

```
public int trap(int[] height) {
    Stack<Integer> stack = new Stack<>();
    int water = 0;

    for (int i = 0; i < height.length; i++) {
        while (!stack.isEmpty() && height[stack.peek()] < height[i]) {
            int bottom = stack.pop();
            if (stack.isEmpty()) break;
            int width = i - stack.peek() - 1;
            int h = Math.min(height[i], height[stack.peek()]) - height[bottom];
            water += width * h;
        }
        stack.push(i);
    }
```

```
        return water;
    }
```

← Previous: Sliding Window | Back to Main Guide | Next: Intervals →